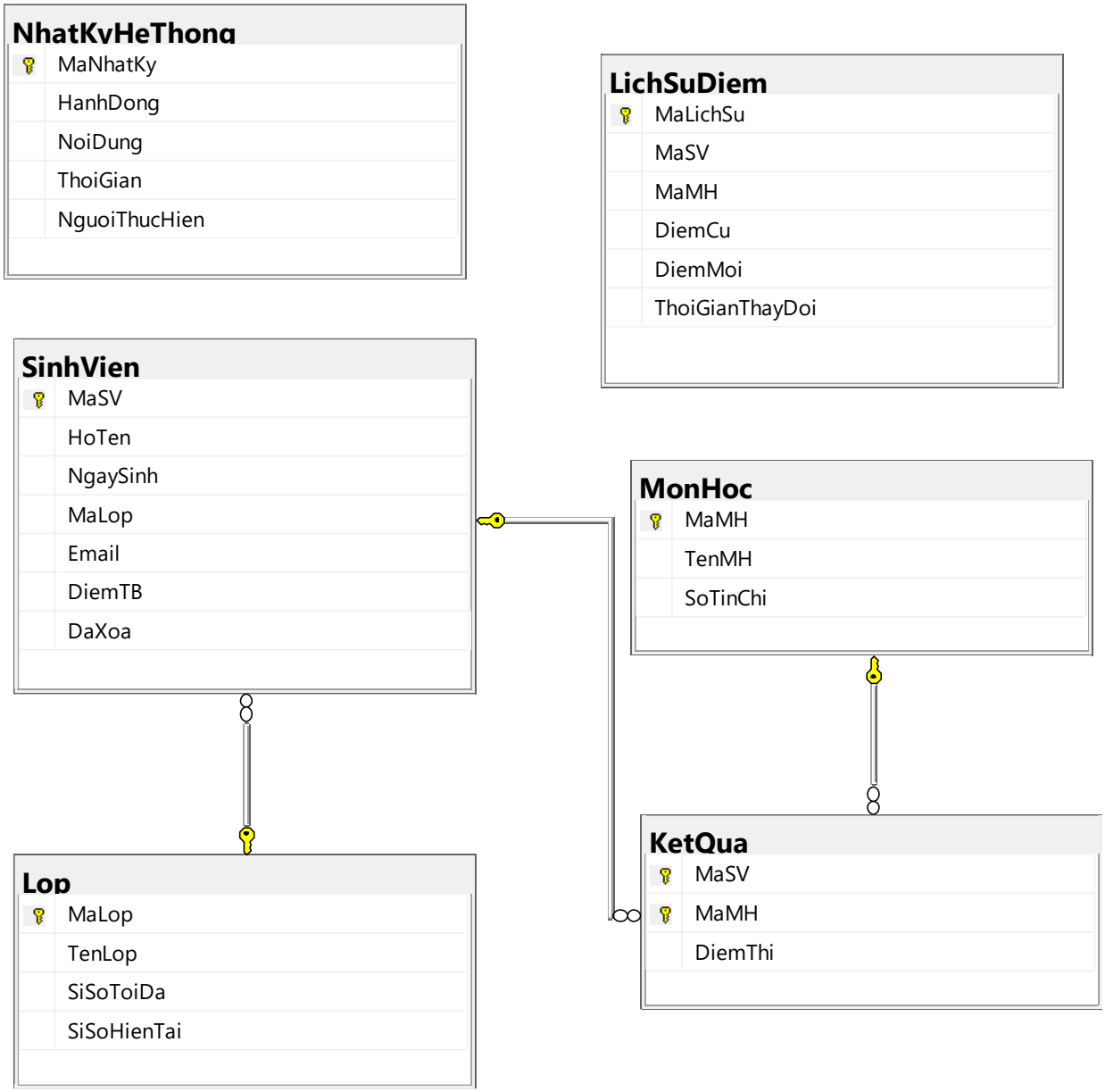
BÀI TẬP

TRIGGER TRONG T-SQL

Cho lược đồ CSDL Quản lý sinh viên như sau:



STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
1	MaLop	CHAR(10)	PK	Mã định danh duy nhất của lớp
2	TenLop	NVARCHAR(50)	NOT NULL	Tên lớp
3	SiSoToiDa	INT	NOT NULL, DEFAULT 40	Sĩ số tối đa của lớp
4	SiSoHienTai	INT	NOT NULL, DEFAULT 0	Số lượng sinh viên hiện tại của lớp

5	MaSV	CHAR(10)	PK	Mã định danh duy nhất của sinh viên
6	HoTen	NVARCHAR(50)	NOT NULL	Họ và tên sinh viên
7	NgaySinh	DATE	NULL	Ngày sinh của sinh viên
8	Email	VARCHAR(100)	NULL	Địa chỉ email của sinh viên
9	DiemTB	DECIMAL(4,2)	DEFAULT 0	Điểm trung bình của sinh viên
10	DaXoa	BIT	NOT NULL, DEFAULT 0	Trạng thái xóa mềm của sinh viên; 0: chưa xóa, 1: đã xóa
11	MaMH	CHAR(10)	PK	Mã định danh duy nhất của môn học
12	TenMH	NVARCHAR(100)	NOT NULL	Tên môn học
13	SoTinChi	INT	NOT NULL, DEFAULT 3	Số tín chỉ của môn học
14	DiemThi	DECIMAL(4,2)	NULL	Điểm thi của sinh viên trong môn học
15	MaNhatKy	INT	PK, IDENTITY(1,1)	Mã định danh duy nhất của bản ghi nhật ký hệ thống
16	HanhDong	NVARCHAR(50)	NULL	Hành động được thực hiện và ghi nhận trong hệ thống
17	NoiDung	NVARCHAR(500)	NULL	Nội dung chi tiết của hành động hoặc sự kiện
18	ThoiGian	DATETIME	DEFAULT GETDATE()	Thời điểm ghi nhận hành động
19	NguoiThucHien	NVARCHAR(100)	DEFAULT SUSER_SNAME()	Tên tài khoản thực hiện thao tác

20	MaLichSu	INT	PK, IDENTITY(1,1)	Mã định danh duy nhất của bản ghi lịch sử điểm
21	DiemCu	DECIMAL(4,2)	NULL	Điểm của sinh viên trước khi thay đổi
22	DiemMoi	DECIMAL(4,2)	NULL	Điểm của sinh viên sau khi thay đổi
23	ThoiGianThayDoi	DATETIME	DEFAULT GETDATE()	Thời điểm điểm được thay đổi

Hãy tạo lập CSDL trên SQL Server và thực hiện các yêu cầu dưới đây

Bài 1.

a) Viết trigger trg_SinhVien_AfterInsert trên bảng SinhVien, kích hoạt sau khi INSERT, ghi một dòng vào NhatKyHeThong với nội dung mô tả sinh viên vừa thêm.

b) Thêm thử một sinh viên và kiểm tra bảng NhatKyHeThong đã có dòng log tương ứng chưa.

c) Giải thích vì sao trigger dùng bảng inserted mà không dùng trực tiếp các biến tham số như trong Stored Procedure.

d) Thử câu lệnh INSERT nhiều dòng cùng lúc (multi-row insert) và kiểm tra trigger có ghi log đủ cho tất cả các dòng hay không.

e) So sánh AFTER trigger với ràng buộc CHECK: trigger có thể thay thế hoàn toàn CHECK constraint được không?

Bài 2.

a) Viết trigger trg_SinhVien_CapNhatSiSo trên SinhVien, sau khi INSERT sẽ tự động tăng SiSoHienTai của lớp tương ứng trong bảng Lop.

b) Thêm 1 sinh viên vào lớp CNTT01 và kiểm tra SiSoHienTai đã tăng đúng.

c) Giải thích vì sao trigger ở câu (a) dùng GROUP BY theo MaLop thay vì chỉ +1 đơn giản.

d) Thử thêm 3 sinh viên cùng lúc vào lớp CNTT01 bằng một câu lệnh INSERT duy nhất, kiểm tra SiSoHienTai có tăng đúng 3 hay không.

e) Nếu cột MaLop trong bảng inserted có giá trị NULL (sinh viên chưa được xếp lớp), trigger ở câu (a) có gây lỗi không? Đề xuất cách xử lý an toàn hơn.

Bài 3.

a) Viết trigger trg_SinhVien_AfterDelete trên SinhVien: sau khi DELETE, giảm SiSoHienTai của lớp tương ứng và ghi log vào NhatKyHeThong.

b) Xóa sinh viên SV007 và kiểm tra cả sĩ số lớp lẫn nhật ký hệ thống.

c) Giải thích bảng ảo deleted khác gì so với bảng inserted, và vì sao trigger DELETE chỉ dùng deleted mà không có inserted.

d) Thử DELETE nhiều sinh viên cùng lúc (bằng điều kiện WHERE khớp nhiều dòng) và kiểm tra sĩ số giảm đúng.

e) Nếu một sinh viên đang có dữ liệu điểm thi trong bảng KetQua, việc DELETE sinh viên đó có gây lỗi không? Vì sao?

Bài 4.

a) Viết trigger trg_SinhVien_AfterUpdate: khi Email của sinh viên bị UPDATE, ghi log lại Email cũ và Email mới vào NhatKyHeThong.

b) Cập nhật Email của sinh viên SV001 và kiểm tra log.

c) Cập nhật HoTen của SV001 (không đổi Email) và giải thích vì sao trigger KHÔNG ghi log lần này.

d) Giải thích vì sao trigger cần JOIN inserted với deleted theo MaSV (thay vì chỉ dùng riêng inserted hoặc riêng deleted).

e) Thử UPDATE nhiều sinh viên cùng lúc (đổi Email của cả SV002 và SV003 trong một câu lệnh) và kiểm tra trigger có ghi log đủ cho cả hai dòng không.

Bài 5.

a) Viết trigger trg_KetQua_LuuLichSu trên bảng KetQua: sau khi UPDATE điểm, lưu điểm cũ/điểm mới vào bảng LichSuDiem.

b) Thêm dữ liệu môn học/điểm mẫu, sau đó sửa điểm của một sinh viên và kiểm tra LichSuDiem.

c) Sửa lại điểm về đúng giá trị cũ (7.5 -> 7.5, không đổi) và kiểm tra trigger có ghi thêm dòng lịch sử thừa không.

d) Sửa trigger để đồng thời TỰ ĐỘNG cập nhật lại cột DiemTB của sinh viên trong bảng SinhVien (tính trung bình cộng đơn giản các điểm hiện có) mỗi khi điểm thay đổi.

e) Giải thích vì sao trigger ở câu (d) phải GROUP BY theo MaSV thay vì tính trung bình cho toàn bộ bảng KetQua, và vì sao chỉ giới hạn trong SinhVien có mặt trong inserted.

Bài 6.

a) Viết trigger trg_KetQua_KiemTraDiem trên KetQua: nếu DiemThi nằm ngoài khoảng 0-10, hủy bỏ (ROLLBACK) toàn bộ thao tác INSERT/UPDATE.

b) Thử INSERT một điểm không hợp lệ (12 điểm) và quan sát kết quả.

c) So sánh cách làm ở bài này (Trigger) với cách dùng CHECK constraint để kiểm tra khoảng điểm 0-10. Nên dùng cách nào?

d) Sửa trigger để, thay vì hủy toàn bộ giao dịch, chỉ ghi log cảnh báo vào NhatKyHeThong và tự động ĐIỀU CHỈNH điểm về giới hạn hợp lệ gần nhất (0 nếu âm, 10 nếu vượt quá).

e) Đây là một dạng ràng buộc mô phỏng khái niệm Assertion đã học ở bài Ràng buộc/Chỉ mục. Giải thích mối liên hệ.

Bài 7.

a) Viết trigger trg_SinhVien_KiemTraSiSo trên SinhVien: trước khi cho thêm sinh viên mới, kiểm tra SiSoHienTai của lớp có vượt quá SiSoToiDa hay không; nếu vượt thì hủy bỏ thao tác.

b) Đặt SiSoToiDa của lớp CNTT01 xuống còn 5, sau đó thử thêm sinh viên thứ 6 để kiểm tra trigger có chặn đúng không.

c) Đây chính là ví dụ ràng buộc liên bảng (tuple/assertion) đã học lý thuyết. Giải thích vì sao KHÔNG THỂ hiện thực ràng buộc này chỉ bằng CHECK constraint thông thường trên bảng SinhVien.

d) Thử nghiệm INSERT nhiều sinh viên cùng lúc khiến lớp vượt sĩ số ngay trong MỘT câu lệnh duy nhất (ví dụ thêm 3 sinh viên vào lớp chỉ còn chỗ cho 1). Trigger có phát hiện đúng không?

e) So sánh việc dùng Trigger để kiểm soát sĩ số (bài này) với việc kiểm tra sĩ số ngay trong Stored Procedure trước khi INSERT (như đã làm ở các bài tập Stored Procedure trước). Ưu nhược điểm của mỗi cách?

Bài 8.

a) Tạo view vw_DangKySinhVien hiển thị thông tin sinh viên kèm tên lớp; sau đó viết trigger INSTEAD OF INSERT trên view này để kiểm soát việc ghi danh qua view.

b) Thử INSERT một sinh viên mới thông qua view vw_DangKySinhVien.

c) Giải thích vì sao cần trigger INSTEAD OF INSERT ở đây: nếu không có trigger này, INSERT trực tiếp vào view có hoạt động được không?

d) Mở rộng trigger để kiểm tra: nếu MaLop được truyền vào không tồn tại trong bảng Lop, hủy bỏ thao tác và báo lỗi rõ ràng.

e) So sánh INSTEAD OF trigger với AFTER trigger: điểm khác biệt cốt lõi là gì?

Bài 9.

a) Viết trigger INSTEAD OF DELETE trên bảng SinhVien: thay vì xóa thật, chỉ đánh dấu DaXoa = 1.

b) Thử "xóa" sinh viên SV012 bằng lệnh DELETE thông thường, sau đó kiểm tra dòng dữ liệu vẫn còn trong bảng nhưng đã đổi DaXoa.

c) Tạo thêm view vw_SinhVienDangHoc chỉ hiển thị các sinh viên chưa bị xóa mềm, để các truy vấn thông thường không cần quan tâm cột DaXoa.

d) Trigger trg_SinhVien_XoaMem ở câu (a) có làm cho trigger trg_SinhVien_AfterDelete và trg_SinhVien_CapNhatSiSo còn hoạt động đúng không? Giải thích.

e) Đề xuất cách viết lại logic cập nhật sĩ số lớp (đã làm ở Bài 3) để vẫn hoạt động đúng khi chuyển sang cơ chế xóa mềm.

Bài 10.

a) Viết trigger trg_KetQua_AfterInsert trên KetQua: sau khi INSERT (có thể nhiều dòng cùng lúc, thuộc nhiều sinh viên khác nhau), tự động tính lại DiemTB cho TẤT CẢ các sinh viên bị ảnh hưởng.

b) Thử INSERT nhiều dòng điểm cho NHIỀU sinh viên khác nhau trong một câu lệnh duy nhất (mô phỏng việc nhập điểm hàng loạt từ file Excel) và kiểm tra DiemTB cập nhật đúng cho tất cả.

c) Giải thích vì sao viết trigger theo kiểu xử lý TẬP HỢP (set-based, dùng GROUP BY/JOIN như câu a) lại tốt hơn nhiều so với việc dùng CURSOR duyệt từng dòng của inserted để cập nhật từng sinh viên một.

d) Thử nghiệm: nếu có 2 dòng KetQua thuộc CÙNG một sinh viên được INSERT trong CÙNG một câu lệnh, trigger có tính đúng điểm trung bình hay không?

e) Đề xuất cách gộp trigger trg_KetQua_AfterInsert (bài này) với trg_KetQua_LuuLichSu (Bài 5, hiện đang là trigger UPDATE riêng) nếu muốn theo dõi cả INSERT lẫn UPDATE trên KetQua trong CÙNG một trigger.

Bài 11.

a) Viết trigger trg_SinhVien_NganXoa trên SinhVien (INSTEAD OF DELETE thay thế cho trigger xóa mềm ở Bài 9, mở rộng thêm điều kiện): nếu sinh viên đang có dữ liệu trong KetQua, không cho xóa mềm và báo lỗi rõ ràng bằng THROW.

b) Thử xóa sinh viên SV001 (đang có điểm môn MH01) và quan sát thông báo lỗi.

c) So sánh THROW với RAISERROR: nên dùng cái nào trong các trigger hiện đại?

d) Bọc thao tác DELETE trong TRY/CATCH ở tầng gọi (client script) để bắt lỗi từ trigger và xử lý một cách 'mềm mại' hơn (ví dụ chỉ in cảnh báo, không dừng chương trình).

e) Phân tích: việc dùng THROW/RAISERROR + ROLLBACK trong trigger có ảnh hưởng gì đến các trigger KHÁC đang chờ thực thi trên cùng bảng (nếu có)?

Bài 12.

a) Viết DDL Trigger trg_DDL_GhiLog ở mức DATABASE, kích hoạt khi có CREATE_TABLE, ALTER_TABLE, hoặc DROP_TABLE, ghi log vào NhatKyHeThong.

b) Thử tạo một bảng tạm để kiểm tra DDL Trigger có ghi log đúng không.

c) Giải thích hàm EVENTDATA() dùng để làm gì trong DDL Trigger, khác gì so với bảng inserted/deleted trong DML Trigger.

d) Viết một DDL Trigger khác NGĂN CHẶN hoàn toàn việc DROP bất kỳ bảng nào trong CSDL (bảo vệ cấu trúc CSDL khỏi bị xóa nhầm).

e) Nếu cần tạm thời cho phép DROP bảng (ví dụ khi bảo trì hệ thống), làm thế nào để vô hiệu hóa DDL Trigger này một cách an toàn?

Bài 13.

a) Viết Logon Trigger trg_Logon_GioHanhChinh ở mức SERVER, chỉ cho phép đăng nhập trong khoảng 6:00 - 22:00, trừ tài khoản sa.

b) Giải thích vì sao Logon Trigger được tạo ở mức ALL SERVER thay vì mức DATABASE như DDL Trigger ở Bài 12.

c) Phân tích rủi ro nghiêm trọng nếu Logon Trigger viết SAI (ví dụ luôn ROLLBACK mọi kết nối), và cách khắc phục khi bị 'khóa' không đăng nhập được nữa.

d) Viết câu lệnh dùng DAC (giả lập, dạng sqlcmd) để kết nối khẩn cấp và vô hiệu hóa Logon Trigger đang gây lỗi.

e) Đề xuất một cách viết Logon Trigger AN TOÀN HƠN, giảm rủi ro tự khóa hệ thống (ví dụ luôn cho phép một nhóm tài khoản quản trị đăng nhập bất kể giờ giấc).

Bài 14.

a) Liệt kê các trigger hiện có trên bảng SinhVien (đã tạo qua các bài trước) và kiểm tra thứ tự thực thi mặc định của chúng.

b) Dùng sp_settriggerorder để quy định trg_SinhVien_CapNhatSiSo (Bài 2) chạy ĐẦU TIÊN (FIRST) trong số các AFTER INSERT trigger trên bảng SinhVien.

c) Dùng sp_settriggerorder để quy định trg_SinhVien_KiemTraSiSo (Bài 7) chạy CUỐI CÙNG (LAST) trong số các AFTER INSERT trigger, đảm bảo nó luôn kiểm tra SAU KHI sĩ số đã được cập nhật.

d) SQL Server chỉ cho phép chỉ định FIRST và LAST, không cho phép quy định thứ tự CHÍNH XÁC ở giữa cho nhiều trigger. Nếu có 3 trigger trở lên cùng sự kiện, làm sao đảm bảo thứ tự thực thi đúng theo ý muốn cho TẤT CẢ?

e) Kiểm tra lại thứ tự thực thi sau khi đã thiết lập bằng sp_settriggerorder.

Bài 15.

a) Xây dựng một tình huống hoàn chỉnh: khi một sinh viên bị xóa mềm, toàn bộ dữ liệu điểm liên quan trong KetQua cũng cần được đánh dấu 'không còn hiệu lực' thay vì bị xóa. Viết trigger phối hợp thực hiện việc này.

b) Liệt kê TOÀN BỘ trigger đã xây dựng xuyên suốt bộ bài tập này và phân loại theo AFTER/INSTEAD OF, DML/DDL/Logon.

c) So sánh 3 công cụ đảm bảo toàn vẹn dữ liệu đã học: CHECK Constraint, Trigger, và Stored Procedure (với kiểm tra thủ công). Lập bảng so sánh theo các tiêu chí: phạm vi áp dụng, thời điểm kiểm tra, khả năng truy vấn liên bảng, hiệu năng, độ phức tạp.

d) Viết một đoạn script tổng kết dùng để VÔ HIỆU HÓA TẤT CẢ trigger trên bảng SinhVien (hữu ích khi cần import dữ liệu hàng loạt mà không muốn kích hoạt logic tự động), và script để KÍCH HOẠT LẠI toàn bộ sau đó.